Aim: A* Search algorithm for solving a pathfinding problem.

```
import heapq

import matplotlib.pyplot as plt

import networkx as nx



# Carol Pillai T104

graph = {

    'MVLU College': {

        'Andheri East': 2.0,

        'Chakala': 2.0,

        'Jogeshwari Caves': 1.8

    },



    'Andheri East':        {'Saki Naka': 3.0},

    'Saki Naka':            {'Ghatkopar Link Rd': 2.6},

    'Ghatkopar Link Rd':    {'LBS Marg': 1.0},
```

```
    'Chakala':                {'Asalpha': 3.0},

    'Asalpha':                {'Sir M V Road': 2.5},

    'Sir M V Road':           {'LBS Marg': 1.0},




    'Jogeshwari Caves':     {'JVLR': 5.3},

    'JVLR':                   {'Powai': 3.5},

    'Powai':                  {'LBS Marg': 2.0},




    'LBS Marg':               {'R City': 0.5},

    'R City': {}

}
```

```
heuristics = {

    'MVLU College': 8.5,

    'Andheri East': 6.0,

    'Chakala': 6.2,

    'Jogeshwari Caves': 9.5,

    'Saki Naka': 3.0,

    'Asalpha': 3.2,

    'JVLR': 6.0,

    'Ghatkopar Link Rd': 1.5,

    'Sir M V Road': 1.6,

    'Powai': 3.0,

    'LBS Marg': 0.8,

    'R City': 0

}



node_positions = {
```

```
    'MVLU College':          (5, 12),

    'Andheri East':          (1, 10),

    'Chakala':               (5, 10),

    'Jogeshwari Caves':      (9, 10),

    'Saki Naka':             (1, 8),

    'Asalpha':               (5, 8),

    'JVLR':                  (9, 8),

    'Ghatkopar Link Rd':     (1, 6),

    'Sir M V Road':          (5, 6),

    'Powai':                 (9, 6),

    'LBS Marg':              (5, 3),

    'R City':                (5, 0)

}




def a_star_search(graph, heuristics, start, goal):
```

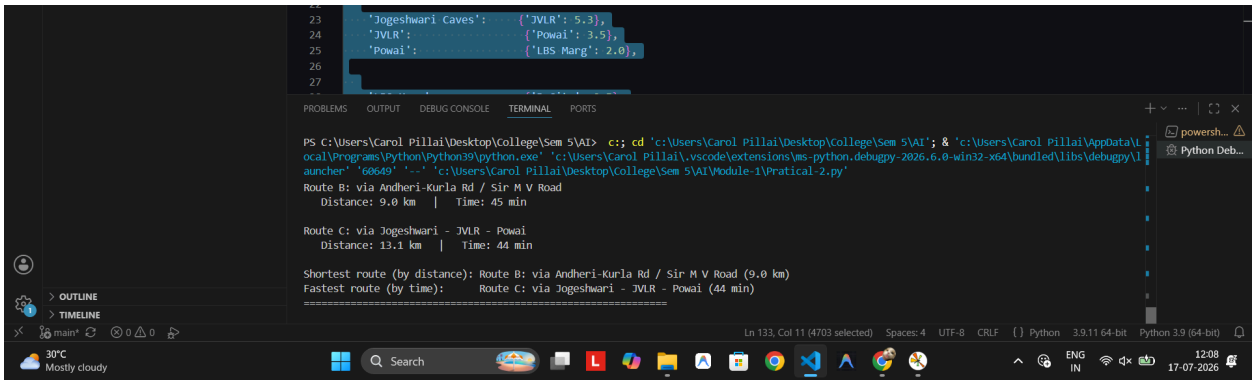
```
    priority_queue = [(heuristics[start], start, [start], 0)]

    visited = set()



    while priority_queue:

        f_score, current, path, g_score = heapq.heappop(priority_queue)

        if current in visited:

            continue

        visited.add(current)

        if current == goal:

            return path, g_score

        for neighbor, edge_weight in graph[current].items():

            if neighbor not in visited:

                next_g = g_score + edge_weight

                next_f = next_g + heuristics[neighbor]

                heapq.heappush(priority_queue, (next_f, neighbor, path +
[neighbor], next_g))
```

```
    return None, float('inf')




optimal_path, total_distance = a_star_search(graph, heuristics, 'MVLU
College', 'R City')

print(f"A* Optimal Path Discovered: {' -> '.join(optimal_path)}")

print(f"Total Road Distance: {round(total_distance, 2)} km\n")







routes = [

    {"name": "Route A: via Andheri - Ghatkopar Link Rd",
"distance_km": 9.1,  "time_min": 45},

    {"name": "Route B: via Andheri-Kurla Rd / Sir M V Road",
"distance_km": 9.0,  "time_min": 45},

    {"name": "Route C: via Jogeshwari - JVLR - Powai",
"distance_km": 13.1, "time_min": 44},

]




shortest = min(routes, key=lambda r: r["distance_km"])
```

```
fastest = min(routes, key=lambda r: r["time_min"])




print("=" * 62)

print("ROUTE COMPARISON: MVLU College -> R City")

print("=" * 62)

for r in routes:

    print(f"{r['name']}\n   Distance: {r['distance_km']} km   |   Time:
{r['time_min']} min\n")




print(f"Shortest route (by distance): {shortest['name']}
({shortest['distance_km']} km)")

print(f"Fastest route (by time):      {fastest['name']}
({fastest['time_min']} min)")

print("=" * 62)







G = nx.DiGraph()

for node, neighbors in graph.items():
```

```
    for neighbor, weight in neighbors.items():

        G.add_edge(node, neighbor, weight=weight)



plt.figure(figsize=(13, 13))



path_edges = list(zip(optimal_path, optimal_path[1:]))

normal_edges = [edge for edge in G.edges() if edge not in path_edges]



nx.draw_networkx_nodes(G, node_positions, node_size=5500,
node_color='lightblue')

nx.draw_networkx_edges(G, node_positions, edgelist=normal_edges,
width=1.5, edge_color='gray', arrows=True,

                       min_source_margin=35, min_target_margin=35)

nx.draw_networkx_edges(G, node_positions, edgelist=path_edges, width=3.5,
edge_color='darkorange', arrows=True,

                       min_source_margin=35, min_target_margin=35)
```

```
node_labels = {node: f"{node}\nh(n)={heuristics[node]}" for node in
G.nodes()}

nx.draw_networkx_labels(G, node_positions, labels=node_labels,
font_size=8, font_weight='bold', font_color='black')



edge_labels = nx.get_edge_attributes(G, 'weight')

formatted_edge_labels = {k: f"{v} km" for k, v in edge_labels.items()}

nx.draw_networkx_edge_labels(G, node_positions,
edge_labels=formatted_edge_labels, font_color='red', font_size=9)



plt.title("A* Routing Map: MVLU College to R City\n(Highlighted Orange
Path = Optimal/Shortest Route)", fontsize=13)

plt.axis('off')

plt.tight_layout()

plt.show()
```


'Jogeshwari Caves': {'JVLR': 5.3},
'JVLR': {'Powai': 3.5},
'Powai': {'LBS Marg': 2.0},
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Carol Pillai\Desktop\College\Sem 5\AI> c:; cd 'c:\Users\Carol Pillai\Desktop\College\Sem 5\AI'; & 'c:\Users\Carol Pillai\AppData\Local\Programs\Python\Python39\python.exe' 'c:\Users\Carol Pillai\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '60649' '--' 'c:\Users\Carol Pillai\Desktop\College\Sem 5\AI\Module-1\Pratical-2.py'
Route B: via Andheri-Kurla Rd / Sir M V Road
Distance: 9.0 km | Time: 45 min
Route C: via Jogeshwari - JVLR - Powai
Distance: 13.1 km | Time: 44 min
Shortest route (by distance): Route B: via Andheri-Kurla Rd / Sir M V Road (9.0 km)
Fastest route (by time): Route C: via Jogeshwari - JVLR - Powai (44 min)
OUTLINE
TIMELINE
Ln 133, Col 11 (4703 selected) Spaces: 4 UTF-8 CRLF Python 3.9.11 64-bit Python 3.9 (64-bit)
30°C Mostly cloudy
Search
ENG IN
12:08 17-07-2026

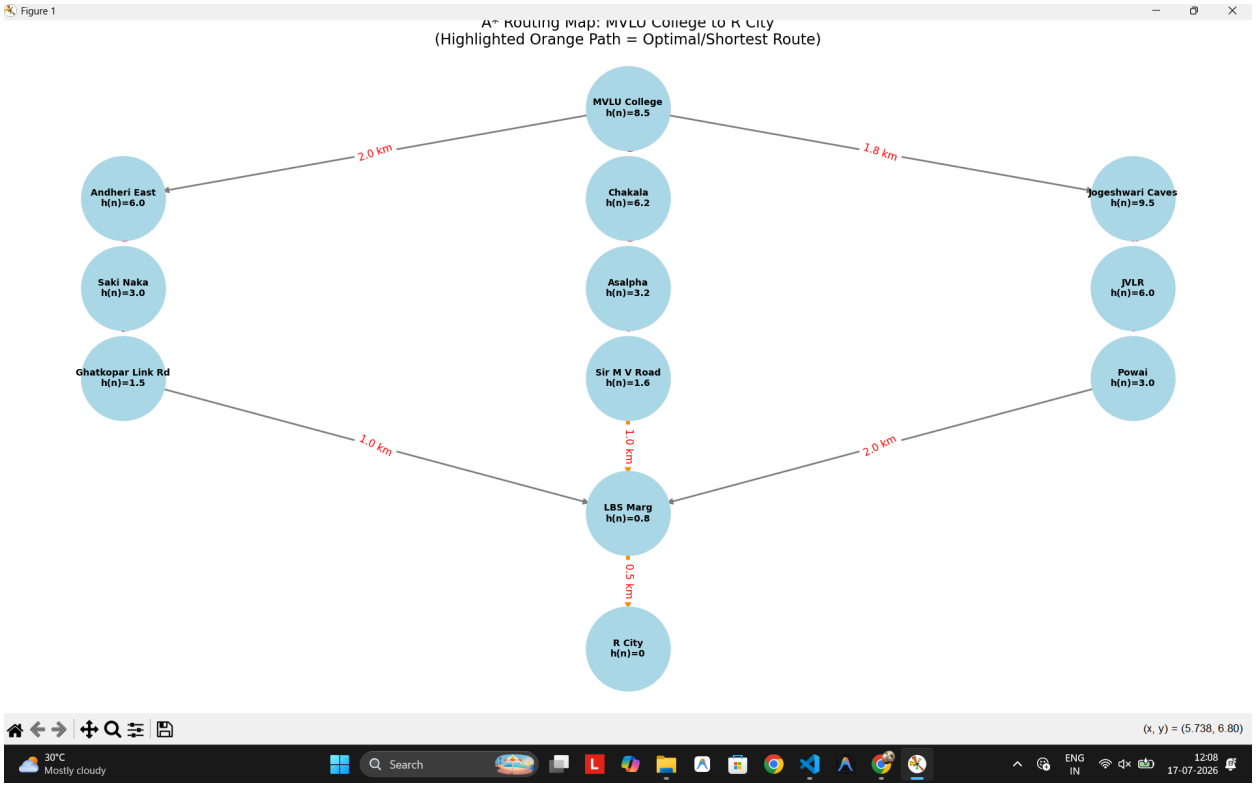


Figure 1
A* Routing Map: MVLU College to R City
(Highlighted Orange Path = Optimal/Shortest Route)
MVLU College h(n)=8.5
Andheri East h(n)=6.0
Chakala h(n)=6.2
Jogeshwari Caves h(n)=9.5
Saki Naka h(n)=3.0
Asalpha h(n)=3.2
JVLR h(n)=6.0
Ghatkopar Link Rd h(n)=1.5
Sir M V Road h(n)=1.6
Powai h(n)=3.0
LBS Marg h(n)=0.8
R City h(n)=0
2.0 km
1.8 km
1.0 km
1.0 km
2.0 km
0.5 km
(x, y) = (5.738, 6.80)
30°C Mostly cloudy
Search
ENG IN
12:08 17-07-2026

Recursive Best-First Search Algorithm (RBFS)

```
import matplotlib.pyplot as plt




#
------------------------------------------------------------------------
--

# 1. NETWORK DEFINITION

#
------------------------------------------------------------------------
--

# Road distances g(n) — same 3 branching routes converging at LBS Marg

graph = {

    'MVLU College': {

        'Andheri East': 2.0,

        'Chakala': 2.0,

        'Jogeshwari Caves': 1.8

    },
```

```
    # Route A: via Andheri - Ghatkopar Link Rd (9.1 km)

    'Andheri East':        {'Saki Naka': 3.0},

    'Saki Naka':            {'Ghatkopar Link Rd': 2.6},

    'Ghatkopar Link Rd':    {'LBS Marg': 1.0},



    # Route B: via Andheri-Kurla Rd / Sir M V Road (9.0 km)

    'Chakala':              {'Asalpha': 3.0},

    'Asalpha':              {'Sir M V Road': 2.5},

    'Sir M V Road':         {'LBS Marg': 1.0},



    # Route C: via Jogeshwari - JVLR - Powai (13.1 km)

    'Jogeshwari Caves':     {'JVLR': 5.3},

    'JVLR':                 {'Powai': 3.5},

    'Powai':                {'LBS Marg': 2.0},
```

```
    'LBS Marg':               {'R City': 0.5},

    'R City': {}

}



# Straight-line distance heuristics h(n) to R City

heuristics = {

    'MVLU College': 8.5,

    'Andheri East': 6.0,

    'Chakala': 6.2,

    'Jogeshwari Caves': 9.5,

    'Saki Naka': 3.0,

    'Asalpha': 3.2,

    'JVLR': 6.0,

    'Ghatkopar Link Rd': 1.5,

    'Sir M V Road': 1.6,
```

```
    'Powai': 3.0,

    'LBS Marg': 0.8,

    'R City': 0

}



# 2D coordinates (X, Y) — MVLU College at top, converging downward to R City

coords = {

    'MVLU College':          (5, 12),

    'Andheri East':          (1, 10),

    'Chakala':               (5, 10),

    'Jogeshwari Caves':      (9, 10),

    'Saki Naka':             (1, 8),

    'Asalpha':               (5, 8),

    'JVLR':                  (9, 8),
```

```
    'Ghatkopar Link Rd':      (1, 6),

    'Sir M V Road':           (5, 6),

    'Powai':                  (9, 6),

    'LBS Marg':               (5, 3),

    'R City':                 (5, 0)

}




#
------------------------------------------------------------------------
--

# 2. RECURSIVE BEST-FIRST SEARCH (RBFS) IMPLEMENTATION

#
------------------------------------------------------------------------
--

def rbfs_search(start, goal):

    # Wrapper function to trigger recursion

    success, path, cost, _ = rbfs(start, goal, g=0,
f_limit=float('inf'), path=[start])
```

```
    return path, cost



def rbfs(node, goal, g, f_limit, path):

    if node == goal:

        return True, path, g, g



    neighbors = graph[node]

    if not neighbors:

        return False, [], 0, float('inf')



    # Construct child nodes list: [computed_f, neighbor_name, next_g]

    successors = []

    for neighbor, distance in neighbors.items():

        if neighbor not in path:  # Prevent cycles

            next_g = g + distance
```

```
            next_f = max(next_g + heuristics[neighbor], g +
heuristics[node])

            successors.append([next_f, neighbor, next_g])



    if not successors:

        return False, [], 0, float('inf')



    while True:

        successors.sort(key=lambda x: x[0])

        best = successors[0]



        if best[0] > f_limit:

            return False, [], 0, best[0]



        alternative_f = successors[1][0] if len(successors) > 1 else
float('inf')
```

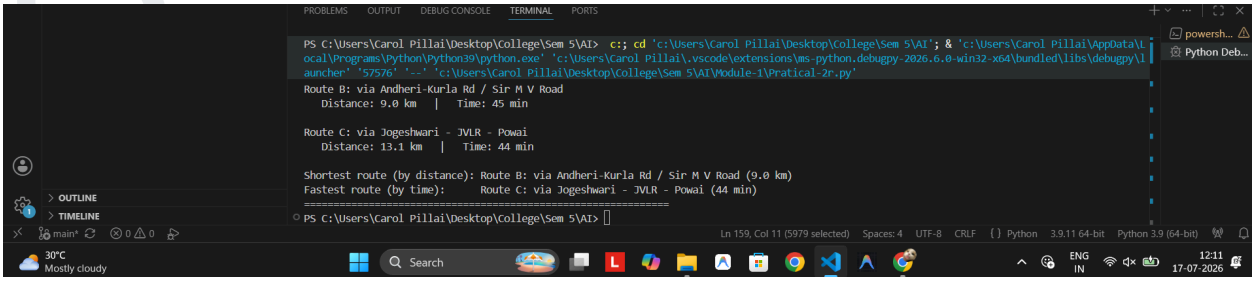
```
        success, result_path, total_g, returned_f = rbfs(
            best[1], goal, best[2], min(f_limit, alternative_f), path + [best[1]]
        )

        best[0] = returned_f

        if success:
            return True, result_path, total_g, returned_f

# Run the algorithm
path, total_dist = rbfs_search('MVLU College', 'R City')
print(f"Optimal RBFS Path: {' -> '.join(path)} ({round(total_dist, 2)} km)")
```

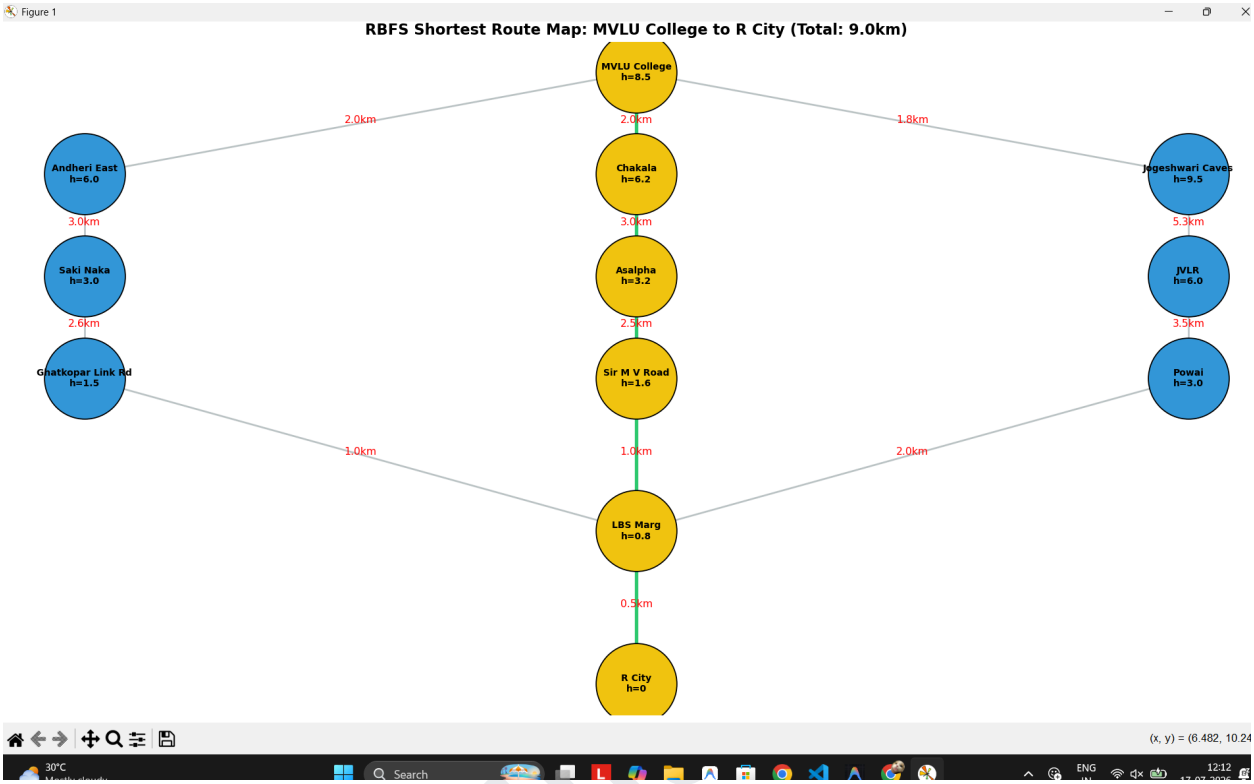
```
#
------------------------------------------------------------------------
--


# 3. ROUTE COMPARISON


#
------------------------------------------------------------------------
--


routes = [


    {"name": "Route A: via Andheri - Ghatkopar Link Rd",
"distance_km": 9.1,  "time_min": 45},


    {"name": "Route B: via Andheri-Kurla Rd / Sir M V Road",
"distance_km": 9.0,  "time_min": 45},


    {"name": "Route C: via Jogeshwari - JVLR - Powai",
"distance_km": 13.1, "time_min": 44},


]




shortest = min(routes, key=lambda r: r["distance_km"])


fastest = min(routes, key=lambda r: r["time_min"])
```

```
print("\n" + "=" * 62)


print("ROUTE COMPARISON: MVLU College -> R City")


print("=" * 62)


for r in routes:


    print(f"{r['name']}\n   Distance: {r['distance_km']} km   |   Time:
{r['time_min']} min\n")




print(f"Shortest route (by distance): {shortest['name']}
({shortest['distance_km']} km)")


print(f"Fastest route (by time):      {fastest['name']}
({fastest['time_min']} min)")


print("=" * 62)




#
----------------------------------------------------------------------
--


# 4. GRAPH PLOTTING (Manual Line Rendering)


#
```

```
------------------------------------------------------------------------
--


plt.figure(figsize=(13, 13))




for node, neighbors in graph.items():

    x1, y1 = coords[node]

    for neighbor, dist in neighbors.items():

        x2, y2 = coords[neighbor]

        is_path = node in path and neighbor in path and
path.index(neighbor) == path.index(node) + 1

        color, width = ('#2ecc71', 3) if is_path else ('#bdc3c7', 1.5)

        plt.plot([x1, x2], [y1, y2], color=color, linewidth=width,
zorder=1)

        plt.text((x1 + x2) / 2, (y1 + y2) / 2, f"{dist}km",
color='red', fontsize=9, ha='center')




for node, (x, y) in coords.items():
```

```
    color = '#f1c40f' if node in path else '#3498db'

    plt.scatter(x, y, color=color, s=5000, zorder=2,
edgecolors='black', linewidths=1)

    plt.text(x, y, f"{node}\nh={heuristics[node]}", ha='center',
va='center',

             color='black', fontsize=8, fontweight='bold')



plt.title(f"RBFS Shortest Route Map: MVLU College to R City (Total:
{round(total_dist, 2)}km)",

          fontsize=13, fontweight='bold')

plt.axis('off')

plt.tight_layout()

plt.show()
```





Carol Pillai
T104